

Apple SDE Intern – Last-Minute Cheatsheet

1. React – Core Mechanics

- **Component** = JS function returning JSX. Must start with **capital letter**.
- **Virtual DOM**: React builds a JS-object tree → **diffs** old vs new → **reconciliation** decides minimal changes → **commit phase** applies to real DOM. Not always faster than direct DOM manipulation for a single known change – it's a general-purpose optimization.
- **State vs Props**: State = internal, mutable via setter, causes re-render. Props = passed from parent, read-only, child re-renders whenever parent re-renders (even if that specific prop didn't change).
- **useState**: plain variables don't persist across re-renders and don't trigger re-renders. `setState` does both – updates value AND schedules re-render.
- **Stale closure gotcha**: `setCount(count+1)` three times in a row = `count+1` once, not +3 (closure captures value at render time). Fix: functional updater `setCount(prev => prev+1)`.
- **Batching**: React 18+ batches multiple `setState` calls into one re-render, everywhere (event handlers, promises, timeouts).
- **useEffect**: runs *after* render is committed to screen (not during render, not before paint). Dependency array: `[]` = once on mount, `[x]` = on mount + when x changes, no array = every render.
- **Cleanup function** (`return () => {...}`): runs before unmount AND before effect re-runs – prevents memory leaks (timers, event listeners, subscriptions).
- **Keys in lists**: React matches old vs new list items by key during diffing. Index as key is fine **only for static lists** (never reordered/filtered/inserted). Prefer stable unique IDs otherwise – wrong key can scramble internal state (e.g. input focus) across reordered items.
- **React.memo** (skip re-render if props unchanged, shallow compare) vs **useMemo** (memoize a computed *value*) vs **useCallback** (memoize a *function reference*, `useCallback(fn, deps) === useMemo(() => fn, deps)`). Overusing these adds overhead for no benefit on cheap computations – use only for genuinely expensive work or breaking `React.memo`.
- **Controlled inputs**: `value={state}` + `onChange={handler}` are a pair. Missing `onChange` = frozen/read-only input.
- **Event handlers**: pass function *reference* (`onClick={handleClick}`), not a call (`onClick={handleClick()}` fires immediately on render).
- **Conditional rendering** `&&`: React renders `null` / `undefined` / `false` / `true` as nothing. Gotcha: `{count && <p/>}` renders literal `"0"` if count is 0 – use `{count > 0 && ...}`.

- **Multi-field forms:** one state object + `{ ...formData, [e.target.name]: e.target.value }`. Never mutate state directly – always create new object/array (reference equality matters for React's re-render detection).

2. Node.js / Express – Core Mechanics

- **Node** = V8 engine outside the browser, given OS access (files, network).
- **http.createServer(cb):** callback runs once **per request**, not once total. Nothing declared inside it persists across requests (like a React component re-running).
- **Express** = convenience layer over `http` – routing (`app.get/post/put/delete`), `res.send/json`, no manual URL/method parsing.
- **Route params** (`/users/:id` → `req.params.id`) = identify a specific resource. **Query params** (`?page=2` → `req.query.page`) = optional filters/pagination, request still valid without them.
- **Middleware:** function (`req, res, next`). Must call `next()` to continue, or request hangs forever. Attach data via `req.user = ...` for downstream handlers to use. Purpose: pull out cross-cutting logic (auth, logging) instead of duplicating per-route.
- **Error-handling middleware:** 4 params (`err, req, res, next`), registered LAST. Triggered via `next(err)`. Centralizes error formatting/logging instead of repeating in every route's catch block.
- **401 vs 403:** 401 = not authenticated (who are you). 403 = authenticated but not authorized (I know you, you can't do this).
- **JWT:** header.payload.signature. Payload is Base64 (readable, NOT encrypted) – never put secrets in it. Signature (made with server-only secret key) prevents tampering – editing payload breaks signature verification. If the secret key ever leaks, attacker can forge any token (e.g. fake admin role).
- **Event loop:** JS is single-threaded. All sync code runs fully first. Then event loop drains the **microtask queue** (Promises/async-await) completely before taking even one task from the **macrotask queue** (setTimeout/setInterval/I/O). `setTimeout(fn, 0) ≠ "run now"` – means "run after current sync code + all microtasks."
- **Concurrency in Node:** comes from NOT blocking on I/O (OS/network handles the actual waiting), not from multiple threads running your JS. `fetch()` calls aren't "on separate threads."
- **Promise.all:** runs concurrently, fails fast (one rejection = whole thing rejects, other results lost). **Promise.allSettled:** waits for all, gives per-promise status – use when partial success is acceptable (e.g. dashboard with 3 independent API calls).

3. DBMS / SQL – Must Know

- **SQL (relational):** fixed schema, tables + foreign keys, ACID guarantees, query planner/optimizer picks efficient execution (index choice, join order).
- **NoSQL (e.g. MongoDB):** flexible/document schema, no default multi-document ACID (single-document ops ARE atomic; multi-doc needs explicit transactions via sessions — heavier, works against typical NoSQL design).
- **ACID:**
 - **Atomicity:** all-or-nothing (e.g. deduct balance + insert stock row — both succeed or both roll back).
 - **Consistency:** DB enforces valid-state rules (e.g. `CHECK (balance >= 0)`).
 - **Isolation:** concurrent transactions don't see each other's intermediate/uncommitted state (prevents lost updates — same root problem as thread race conditions, solved via locking/MVCC instead of mutexes).
 - **Durability:** once committed, survives crashes (written to disk/WAL).
- **NOT YET COVERED — review before interview:**
 - **Joins:** INNER (only matching rows both sides), LEFT (all left + matched right, NULL if none), RIGHT, FULL OUTER. Be ready to write one from memory.
 - **Indexing:** speeds up reads (B-tree typically) at the cost of slower writes (index must update too) and extra storage. Index columns used in WHERE/JOIN/ORDER BY frequently.
 - **Normalization:** 1NF/2NF/3NF — reducing redundancy/anomalies by splitting tables + relations. Trade-off: normalized = less redundancy but more joins; denormalized = faster reads, more duplication.
 - **Primary key vs foreign key vs unique key** — be crisp on definitions.

4. OOP — NOT YET COVERED, review independently

Be ready with a definition + a concrete example for each (use your HPE C++ code or a simple class):

- **Encapsulation:** bundling data + methods, restricting direct access (private members, getters/setters).
- **Abstraction:** exposing only relevant behavior, hiding implementation detail (e.g. `ConnectionPool.get_connection()` hides pool internals).
- **Inheritance:** child class reuses/extends parent class behavior.
- **Polymorphism:** same interface, different underlying behavior (function overloading = compile-time, virtual functions/overriding = run-time).

5. CS Fundamentals — Covered, Key Points to Restate Fast

- **Process vs Thread:** process = independent memory space; thread = lightweight, shares memory within a process → faster context switch (only save stack/registers, not full memory/code/data).
- **Race condition:** unsynchronized concurrent read/write to shared memory → lost updates (classic example: two threads reading same balance before either writes back).
- **Deadlock:** needs ALL 4 Coffman conditions – mutual exclusion, hold-and-wait, no preemption, circular wait. Note: threads all waiting on the *same* resource type is starvation, not deadlock (no circular wait).
- **Mutex + condition variable:** mutex = one thread in critical section at a time. Condition variable = sleep instead of busy-wait until a resource is available (used in your ConnectionPool/ThreadPool).
- **RAII:** resource cleanup tied to object lifetime/destructor – guarantees release even if exception is thrown mid-function (your `ConnectionGuard` pattern).

6. Security Basics – Skim Before Interview

- **CORS:** browser blocks cross-origin requests by default; server must explicitly allow origins via headers. (Your resume mentions fixing CORS issues – be ready to explain what a misconfiguration looked like.)
- **Row Level Security (Supabase/Postgres):** DB-level authorization – even if application code has a filtering bug, Postgres itself restricts rows per user via policy (`auth.uid() = user_id`). Different layer from Express auth middleware (defense in depth).
- **bcrypt:** hashes passwords one-way (never store plaintext); used in Quick-Pick.
- **HTTP-only cookies:** JS can't read them → mitigates XSS-based token theft (vs. storing JWT in localStorage).
- **SQL/NoSQL injection** – know it exists, know parameterized queries / ORM methods prevent it (haven't deep-dived, mention if asked: "I use the Supabase client / Mongoose which parameterizes queries, not raw string concatenation").

7. Your Projects – Quick Recall

Invest-Tracker (React, Node/Express, Supabase/Postgres):

- Auth via Supabase (issues/manages JWT – you verify it in middleware via `supabase.auth.getUser(token)` , you don't hand-sign it).
- API fallback: on add-stock, fallback to buy price if live price fetch fails. On periodic refresh (every 5 min), skip update on failure, keep last known price – graceful degradation, no broken/zeroed data shown.
- RLS + Express middleware = two independent layers (auth vs authorization).

Quick-Pick (React/Vite, Node/Express, MongoDB):

- Hand-rolled JWT (`jsonwebtoken` + bcrypt), stored in HTTP-only cookie.

- `authMiddleware` (valid JWT) + `adminMiddleware` (role === "admin") — **you found and fixed a real gap where these weren't applied to admin routes** — be honest this was found/fixed during interview prep, not originally.
- Known unfixed gaps (be ready to discuss honestly if asked): checkout (`capturePayment`) does stock deduction + cart deletion + order save as separate un-transactional MongoDB writes — no atomicity guarantee if server crashes mid-checkout; also no stock-availability check before deducting (possible overselling under concurrent purchases); a null-reference bug in the "not enough stock" error path.
- PayPal integration for checkout flow.

HPE (Career Preview Program) — C++ payment gateway PoC:

- Owned Authorization Service: Kafka consumer → validates against Postgres → publishes result event.
- Bottleneck: Postgres connection contention under load → fixed with custom **ThreadPool** + **ConnectionPool** (mutex + condition variable, blocking wait not busy-wait).
- **ConnectionGuard** = RAII wrapper — connection auto-returned to pool via destructor even if exception thrown mid-query.
- Lesson to state explicitly: "designing for concurrency matters as much as correctness."

8. DSA — Your Strength, Just Practice Explaining Out Loud

- You're Codeforces Specialist — the algorithms aren't the risk, **narrating your thought process clearly under pressure** is what to practice.
- Structure every answer: brute force → complexity → optimization → complexity → edge cases → walk through an example.
- Practice stating time/space complexity out loud immediately after coding, unprompted.
- Two Sum pattern (hash map $O(n)/O(n)$ vs sorted two-pointer $O(n \log n)/O(1)$, index-recovery pitfall after sorting) — you've already nailed this one, use it as a confidence anchor.

9. Not Touched At All — If Time Permits

- **System design basics** (even at intern level, sometimes a light version comes up): load balancing, caching, horizontal vs vertical scaling — at least know the vocabulary.
- **REST API design principles** beyond what's covered: idempotency (GET/PUT/DELETE should be idempotent, POST isn't), proper status codes (200/201/204/400/401/403/404/500).
- **Git basics**: merge vs rebase, resolving conflicts — standard SDE intern question.
- **Testing**: unit vs integration test, have you written any tests for these projects (honest answer if none: say so, and what you'd add).

- **Big-O of common data structure operations** – array/list/hashmap/tree, insert/delete/search – have this instantly ready.

10. Delivery Reminders

- Past tense, consistently, for completed project work.
- Lead with CP → HPE → Invest-Tracker → Quick-Pick in intro; keep to ~90 seconds.
- If you don't know something: say so plainly, then reason out loud from what you DO know – this scored well with the interviewer persona throughout this prep, it'll work live too.
- For the Quick-Pick admin-route fix and checkout gaps: honesty ("I found this while prepping and fixed/am aware of it") lands far better than pretending it was always solid.